

Dylan Carlson Swing Follow-Through Analysis

Jackson Reel

STAT 463 Final Project

Contents

Introduction	1
Data And Labeling	2
Original Descriptive Results	6
Contact Quality	8
Contact Quality On Pitches In The Strike Zone	10
Alternative Explanations Checked	12
Two-Strike Counts	12
Chase Pitches And Pitch Location	13
Pitch Velocity	15
Which Statcast Variables Explain Follow-Through?	17
Drop-One Importance	18
Coefficient View	19
Cross-Validated Permutation Importance	21
Interpretation	23
Does Follow-Through Still Matter After Statcast Context?	23
Do The Context Variables Explain Away The Difference?	26
Image-Based Machine Learning Attempt	30
Video Linking and Frame Extraction	31
Dataset Construction	31
Exploratory Model Development	33
Machine Learning Lessons	33
Future Computer Vision Work	34
Conclusion	34
Appendix: Reproducibility Notes	34

Introduction

This project studies Dylan Carlson's swing follow-through during the 2023 MLB season. I chose Carlson because I am a Cardinals fan and noticed, just from watching games, that his follow-through seemed to vary in a meaningful way. The purpose of the project is to test whether that visual observation had any statistical validity.

The original version of this project was built around a manually labeled data set: for each Carlson swing, I watched the associated video and coded whether his follow-through finished with one hand, two hands, a check swing, a bunt, or an unusable label. The motivating baseball question is simple:

When Carlson finishes with one hand instead of two, is that just a symptom of pitch context, or is it connected to worse swing quality even after accounting for context?

The first version of the project found a large descriptive difference between one-hand and two-hand follow-throughs, especially when Carlson hit left-handed. This expanded version keeps that baseball question, but makes the statistical goal more explicit: use Statcast variables to determine which pitch, count, and contact variables are important for explaining follow-through choice and batted-ball quality.

There are two related but separate questions in the project. The statistical analysis asks which Statcast variables help explain Carlson's follow-through choice and whether one follow-through is associated with better contact quality. The machine learning work asks whether the manual follow-through labels could eventually be automated from video. The machine learning portion is exploratory and is not used as evidence for the main baseball conclusions.

Because no public Statcast field records follow-through type, the follow-through labels are the original contribution of the project. A second goal was to test whether video-based machine learning could automate those labels, making the project expandable to more players. That computer vision effort became an important part of the final project even though it did not produce a reliable classifier.

The limitation is equally important: the sample is one player and one season. The analysis should be read as a careful player case study, not a league-wide causal claim.

Data And Labeling

The main analysis uses `carlson_swings_labeled.csv`, which joins Baseball Savant pitch-level data to my manually collected follow-through labels using MLB `playIds`. The file `savant_with_playids.csv` contains the same Statcast data before the follow-through labels were joined.

The one-hand versus two-hand distinction is manually defined and somewhat arbitrary. Any swing where Carlson releases one hand from the bat is counted as a one-hand follow-through. If both hands stay on the bat through the visible follow-through, it is counted as a two-hand follow-through. This definition is intentionally simple so that it can be applied consistently while watching video, but it should still be understood as a human-coded label rather than an official Statcast measurement.

```
raw_swings = read_csv("carlson_swings_labeled.csv", show_col_types = FALSE)

glimpse(raw_swings)

## Rows: 482
## Columns: 120
## $ pitch_type           <chr> "FF", "FF", "CH", "SI", "FC", ~
## $ game_date            <date> 2023-08-09, 2023-08-09, 2023~
## $ release_speed        <dbl> 94.6, 95.4, 88.8, 91.0, 91.3, ~
## $ release_pos_x        <dbl> 0.97, -1.96, 1.05, -1.91, -1.~
## $ release_pos_z        <dbl> 5.82, 5.58, 5.91, 3.75, 5.74, ~
## $ player_name          <chr> "Carlson, Dylan", "Carlson, D~
## $ batter               <dbl> 666185, 666185, 666185, 66618~
## $ pitcher              <dbl> 656222, 542888, 656222, 68733~
## $ events               <chr> NA, "field_out", NA, NA, NA, ~
## $ description          <chr> "foul", "hit_into_play", "swi~
## $ spin_dir             <lgl> NA, NA, NA, NA, NA, NA, NA, N~
## $ spin_rate_deprecated <lgl> NA, NA, NA, NA, NA, NA, NA, N~
## $ break_angle_deprecated <lgl> NA, NA, NA, NA, NA, NA, NA, N~
```

```

## $ break_length_deprecated
## $ zone
## $ des
## $ game_type
## $ stand
## $ p_throws
## $ home_team
## $ away_team
## $ type
## $ hit_location
## $ bb_type
## $ balls
## $ strikes
## $ game_year
## $ pfx_x
## $ pfx_z
## $ plate_x
## $ plate_z
## $ on_3b
## $ on_2b
## $ on_1b
## $ outs_when_up
## $ inning
## $ inning_topbot
## $ hc_x
## $ hc_y
## $ tfs_deprecated
## $ tfs_zulu_deprecated
## $ umpire
## $ sv_id
## $ vx0
## $ vy0
## $ vz0
## $ ax
## $ ay
## $ az
## $ sz_top
## $ sz_bot
## $ hit_distance_sc
## $ launch_speed
## $ launch_angle
## $ effective_speed
## $ release_spin_rate
## $ release_extension
## $ game_pk
## $ fielder_2
## $ fielder_3
## $ fielder_4
## $ fielder_5
## $ fielder_6
## $ fielder_7
## $ fielder_8
## $ fielder_9
## $ release_pos_y

<lg1> NA, NA, NA, NA, NA, NA, NA, N~
<dbl> 12, 1, 4, 5, 14, 14, 8, 14, 8~
<chr> "Foul", "Dylan Carlson flies ~
<chr> "R", "R", "R", "R", "R", "R",~
<chr> "R", "L", "R", "L", "L", "L",~
<chr> "L", "R", "L", "R", "R", "R",~
<chr> "TB", "TB", "TB", "TB", "TB",~
<chr> "STL", "STL", "STL", "STL", "~
<chr> "S", "X", "S", "S", "S", "S",~
<dbl> NA, 7, NA, NA, NA, NA, NA, 6,~
<chr> NA, "fly_ball", NA, NA, NA, N~
<dbl> 2, 1, 1, 1, 1, 0, 0, 1, 1, 1,~
<dbl> 2, 2, 1, 1, 2, 2, 1, 2, 2, 2,~
<dbl> 2023, 2023, 2023, 2023, 2023,~
<dbl> 0.40, -0.53, 1.01, -1.41, 0.2~
<dbl> 1.32, 1.18, 1.01, -0.13, 0.60~
<dbl> 0.51, -0.35, -0.36, 0.19, 1.1~
<dbl> 3.77, 2.89, 2.25, 2.33, 1.85,~
<dbl> 691023, NA, 691023, NA, NA, N~
<dbl> NA, NA, NA, NA, NA, NA, NA, N~
<dbl> NA, NA, NA, NA, NA, NA, NA, 6~
<dbl> 2, 1, 2, 0, 1, 1, 1, 2, 2, 2,~
<dbl> 2, 6, 2, 4, 6, 6, 6, 8, 8, 8,~
<chr> "Top", "Top", "Top", "Top", "~
<dbl> NA, 56.66, NA, NA, NA, NA, NA~
<dbl> NA, 139.30, NA, NA, NA, NA, N~
<lg1> NA, NA, NA, NA, NA, NA, NA, N~
<lg1> NA, NA, NA, NA, NA, NA, NA, N~
<lg1> NA, NA, NA, NA, NA, NA, NA, N~
<lg1> NA, NA, NA, NA, NA, NA, NA, N~
<dbl> -2.120857, 5.473731, -5.57079~
<dbl> -137.8013, -138.8994, -129.24~
<dbl> -2.6360443, -4.1828560, -5.04~
<dbl> 5.6118785, -8.0209874, 12.488~
<dbl> 29.34352, 29.70849, 26.59383,~
<dbl> -15.00611, -16.11013, -20.022~
<dbl> 3.39, 3.39, 3.39, 3.39, 3.39,~
<dbl> 1.64, 1.64, 1.64, 1.64, 1.64,~
<dbl> NA, 227, NA, 208, 5, 130, NA,~
<dbl> NA, 89.7, NA, 75.0, 57.2, 77.~
<dbl> NA, 59, NA, 52, -20, 24, NA, ~
<dbl> 95.2, 96.4, 88.8, 93.3, 91.9,~
<dbl> 2096, 2422, 1789, 2034, 2594,~
<dbl> 6.5, 6.7, 6.2, 7.6, 6.6, 6.6,~
<dbl> 717064, 717064, 717064, 71706~
<dbl> 650907, 650907, 650907, 65090~
<dbl> 650490, 650490, 650490, 65049~
<dbl> 664040, 664040, 664040, 66404~
<dbl> 670623, 670623, 670623, 67062~
<dbl> 677551, 677551, 677551, 67755~
<dbl> 668227, 668227, 668227, 66822~
<dbl> 642350, 642350, 642350, 64235~
<dbl> 670042, 670042, 670042, 67004~
<dbl> 53.99, 53.77, 54.26, 52.86, 5~

```

```

## $ estimated_ba_using_speedangle <dbl> NA, 0.016, NA, NA, NA, NA, NA~
## $ estimated_woba_using_speedangle <dbl> NA, 0.021, NA, NA, NA, NA, NA~
## $ woba_value <dbl> NA, 0.00, NA, NA, NA, NA, NA,~
## $ woba_denom <dbl> NA, 1, NA, NA, NA, NA, NA, 1,~
## $ babip_value <dbl> NA, 0, NA, NA, NA, NA, NA, 0,~
## $ iso_value <dbl> NA, 0, NA, NA, NA, NA, NA, 0,~
## $ launch_speed_angle <dbl> NA, 3, NA, NA, NA, NA, NA, 2,~
## $ at_bat_number <dbl> 12, 49, 12, 31, 49, 49, 49, 6~
## $ pitch_number <dbl> 5, 6, 3, 3, 5, 3, 2, 6, 5, 4,~
## $ pitch_name <chr> "4-Seam Fastball", "4-Seam Fa~
## $ home_score <dbl> 0, 3, 0, 2, 3, 3, 3, 4, 4, 4,~
## $ away_score <dbl> 1, 5, 1, 3, 5, 5, 5, 6, 6, 6,~
## $ bat_score <dbl> 1, 5, 1, 3, 5, 5, 5, 6, 6, 6,~
## $ fld_score <dbl> 0, 3, 0, 2, 3, 3, 3, 4, 4, 4,~
## $ post_away_score <dbl> 1, 5, 1, 3, 5, 5, 5, 6, 6, 6,~
## $ post_home_score <dbl> 0, 3, 0, 2, 3, 3, 3, 4, 4, 4,~
## $ post_bat_score <dbl> 1, 5, 1, 3, 5, 5, 5, 6, 6, 6,~
## $ post_fld_score <dbl> 0, 3, 0, 2, 3, 3, 3, 4, 4, 4,~
## $ if_fielding_alignment <chr> "Standard", "Standard", "Stan~
## $ of_fielding_alignment <chr> "Standard", "Standard", "Stan~
## $ spin_axis <dbl> 158, 214, 134, 248, 175, 215,~
## $ delta_home_win_exp <dbl> 0.000, 0.010, 0.007, 0.005, 0~
## $ delta_run_exp <dbl> 0.000, -0.189, -0.061, -0.061~
## $ bat_speed <dbl> 45.6, 70.3, NA, 69.5, 64.9, 6~
## $ swing_length <dbl> 5.1, 6.8, NA, 6.9, 7.3, 7.1, ~
## $ estimated_slg_using_speedangle <dbl> NA, 0.032, NA, NA, NA, NA, NA~
## $ delta_pitcher_run_exp <dbl> 0.000, 0.189, 0.061, 0.061, 0~
## $ hyper_speed <dbl> NA, 89.7, NA, 88.0, 88.0, 88.~
## $ home_score_diff <dbl> -1, -2, -1, -1, -2, -2, -2, --
## $ bat_score_diff <dbl> 1, 2, 1, 1, 2, 2, 2, 2, 2,~
## $ home_win_exp <dbl> 0.411, 0.241, 0.409, 0.378, 0~
## $ bat_win_exp <dbl> 0.589, 0.759, 0.591, 0.622, 0~
## $ age_pit_legacy <dbl> 29, 32, 29, 25, 32, 32, 32, 2~
## $ age_bat_legacy <dbl> 24, 24, 24, 24, 24, 24, 24, 2~
## $ age_pit <dbl> 30, 33, 30, 26, 33, 33, 33, 2~
## $ age_bat <dbl> 25, 25, 25, 25, 25, 25, 25, 2~
## $ n_thruorder_pitcher <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ n_priorpa_thisgame_player_at_bat <dbl> 0, 2, 0, 1, 2, 2, 2, 3, 3, 3,~
## $ pitcher_days_since_prev_game <dbl> 36, 3, 36, 4, 3, 3, 3, 3, 3, ~
## $ batter_days_since_prev_game <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ pitcher_days_until_next_game <dbl> 23, 3, 23, 3, 3, 3, 3, 2, 2, ~
## $ batter_days_until_next_game <dbl> NA, NA, NA, NA, NA, NA, NA, NA, N~
## $ api_break_z_with_gravity <dbl> 1.22, 1.31, 1.88, 2.88, 2.13,~
## $ api_break_x_arm <dbl> 0.40, 0.53, 1.01, 1.41, -0.26~
## $ api_break_x_batter_in <dbl> -0.40, -0.53, -1.01, -1.41, 0~
## $ arm_angle <dbl> 48.3, 41.3, 44.5, 0.1, 41.1, ~
## $ attack_angle <dbl> -7.310456, 12.562193, NA, 10.~
## $ attack_direction <dbl> 46.0060065, 1.7718774, NA, 0.~
## $ swing_path_tilt <dbl> 27.00531, 31.14493, NA, 43.25~
## $ intercept_ball_minus_batter_pos_x_inches <dbl> 40.50031, 35.44065, NA, 31.92~
## $ intercept_ball_minus_batter_pos_y_inches <dbl> 4.611832, 28.657148, NA, 28.9~
## $ playId <chr> "be9a1523-6474-44ff-9614-731c~
## $ `Hands on Swing` <chr> "Check", "2", "2", "2", "1", ~

```

```

swings = raw_swings %>%
  mutate(
    follow_raw = as.character(`Hands on Swing`),
    follow_type = case_when(
      follow_raw == "1" ~ "One hand",
      follow_raw == "2" ~ "Two hands",
      follow_raw == "Check" ~ "Check",
      follow_raw == "Bunt" ~ "Bunt",
      TRUE ~ "Other"
    ),
    full_swing = follow_type %in% c("One hand", "Two hands"),
    one_hand = follow_type == "One hand",
    two_hand = follow_type == "Two hands",
    batter_side = factor(stand, levels = c("L", "R"),
                        labels = c("Batting left", "Batting right")),
    pitcher_hand = factor(p_throws, levels = c("L", "R"),
                        labels = c("LHP", "RHP")),
    two_strikes = strikes == 2,
    in_zone = zone %in% 1:9,
    chase = !in_zone,
    pitch_group = case_when(
      pitch_type %in% c("FF", "SI", "FC", "FA") ~ "Fastball/cutter/sinker",
      pitch_type %in% c("SL", "ST", "CU", "KC", "SV") ~ "Breaking",
      pitch_type %in% c("CH", "FS", "FO") ~ "Offspeed",
      TRUE ~ "Other"
    ),
    pitch_group = factor(pitch_group),
    contact = description == "hit_into_play" | !is.na(launch_speed),
    hard_hit = launch_speed >= 95,
    sweet_spot = launch_angle >= 10 & launch_angle <= 30,
    productive_contact = hard_hit & sweet_spot,
    count_state = paste0(balls, "-", strikes)
  )

full_swings = swings %>%
  filter(full_swing) %>%
  mutate(
    follow_type = factor(follow_type, levels = c("Two hands", "One hand")),
    one_hand = follow_type == "One hand",
    zone = factor(zone)
  )

full_swings %>%
  count(follow_type) %>%
  mutate(rate = n / sum(n)) %>%
  kable(digits = 3, caption = "Follow-through labels among full swings")

```

Table 1: Follow-through labels among full swings

follow_type	n	rate
Two hands	286	0.622
One hand	174	0.378

The modeling data excludes check swings, bunts, and the single unusable label because they are not relevant to the main question: why does Carlson have different follow-throughs on full swings, and is one better than the other? Including bunts or check swings would mix fundamentally different baseball actions into the same outcome. This leaves 460 full swings: 174 one-hand follow-throughs and 286 two-hand follow-throughs.

Original Descriptive Results

The original project showed that follow-through type is strongly related to batting side.

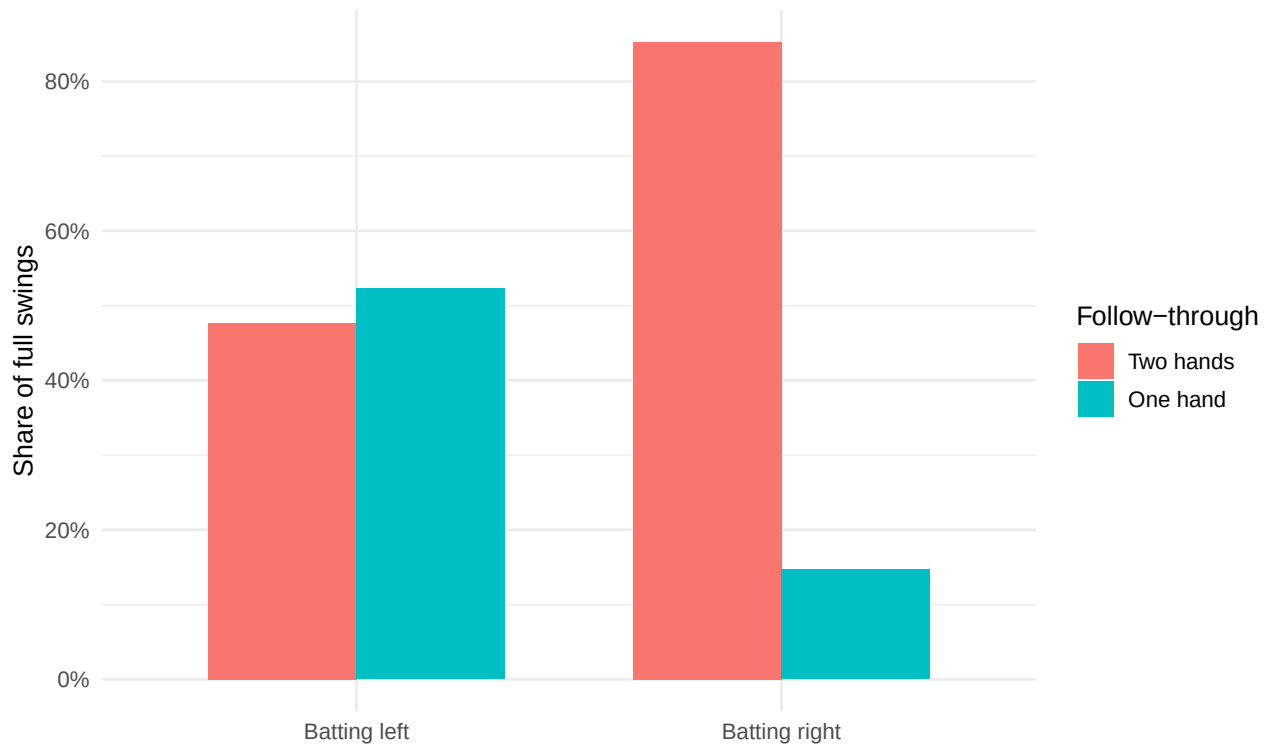
```
full_swings %>%
  count(batter_side, follow_type) %>%
  group_by(batter_side) %>%
  mutate(rate = n / sum(n)) %>%
  ungroup() %>%
  kable(digits = 3, caption = "Follow-through type by batting side")
```

Table 2: Follow-through type by batting side

batter_side	follow_type	n	rate
Batting left	Two hands	135	0.477
Batting left	One hand	148	0.523
Batting right	Two hands	151	0.853
Batting right	One hand	26	0.147

```
full_swings %>%
  count(batter_side, follow_type) %>%
  group_by(batter_side) %>%
  mutate(rate = n / sum(n)) %>%
  ggplot(aes(x = batter_side, y = rate, fill = follow_type)) +
  geom_col(position = "dodge", width = 0.7) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    x = NULL,
    y = "Share of full swings",
    fill = "Follow-through",
    title = "Carlson's follow-through usage differed sharply by batting side"
  )
```

Carlson's follow-through usage differed sharply by batting side



Two points matter for the rest of the project:

1. Carlson was much more likely to finish with two hands when batting right-handed.
2. The left-handed sample is the key baseball problem, because one-hand and two-hand swings were both common enough to compare.

```
full_swings %>%
  mutate(
    result_group = case_when(
      description %in% c("swinging_strike", "swinging_strike_blocked") |
        events == "strikeout" ~ "Miss or strikeout",
      description %in% c("foul", "foul_tip") ~ "Foul",
      description == "hit_into_play" ~ "Ball in play",
      TRUE ~ "Other"
    )
  ) %>%
  count(batter_side, follow_type, result_group) %>%
  group_by(batter_side, follow_type) %>%
  mutate(rate = n / sum(n)) %>%
  ungroup() %>%
  arrange(batter_side, follow_type, desc(rate)) %>%
  kable(digits = 3, caption = "Swing outcomes by batting side and follow-through")
```

Table 3: Swing outcomes by batting side and follow-through

batter_side	follow_type	result_group	n	rate
Batting left	Two hands	Foul	66	0.489
Batting left	Two hands	Ball in play	52	0.385
Batting left	Two hands	Miss or strikeout	17	0.126

batter_side	follow_type	result_group	n	rate
Batting left	One hand	Ball in play	60	0.405
Batting left	One hand	Foul	50	0.338
Batting left	One hand	Miss or strikeout	38	0.257
Batting right	Two hands	Foul	70	0.464
Batting right	Two hands	Ball in play	54	0.358
Batting right	Two hands	Miss or strikeout	27	0.179
Batting right	One hand	Foul	13	0.500
Batting right	One hand	Miss or strikeout	7	0.269
Batting right	One hand	Ball in play	6	0.231

Contact Quality

For swings where Statcast records batted-ball information, two-hand follow-throughs produced better contact quality in the original analysis. The table below updates that result using the joined Statcast file directly.

```
contact_swings = full_swings %>%
  filter(!is.na(launch_speed), !is.na(launch_angle))

contact_swings %>%
  group_by(batter_side, follow_type) %>%
  summarize(
    n = n(),
    avg_ev = mean(launch_speed, na.rm = TRUE),
    median_ev = median(launch_speed, na.rm = TRUE),
    hard_hit_rate = mean(hard_hit, na.rm = TRUE),
    avg_la = mean(launch_angle, na.rm = TRUE),
    sweet_spot_rate = mean(sweet_spot, na.rm = TRUE),
    productive_contact_rate = mean(productive_contact, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  kable(digits = 3, caption = "Batted-ball quality by batting side and follow-through")
```

Table 4: Batted-ball quality by batting side and follow-through

batter_side	follow_type	n	avg_ev	median_ev	hard_hit_rate	avg_la	sweet_spot_rate	productive_contact_rate
Batting left	Two hands	106	85.945	84.90	0.302	30.811	0.292	0.142
Batting left	One hand	102	81.422	81.75	0.118	9.304	0.206	0.039
Batting right	Two hands	119	82.839	83.20	0.244	26.672	0.261	0.109
Batting right	One hand	18	70.661	72.10	0.000	9.056	0.111	0.000

```
contact_swings %>%
  ggplot(aes(x = follow_type, y = launch_speed, fill = follow_type)) +
  geom_boxplot(alpha = 0.75, outlier.alpha = 0.45) +
  facet_wrap(~ batter_side) +
  labs(
    x = NULL,
```

```

y = "Exit velocity",
fill = "Follow-through",
title = "Two-hand follow-throughs generated stronger contact"
) +
guides(fill = "none")

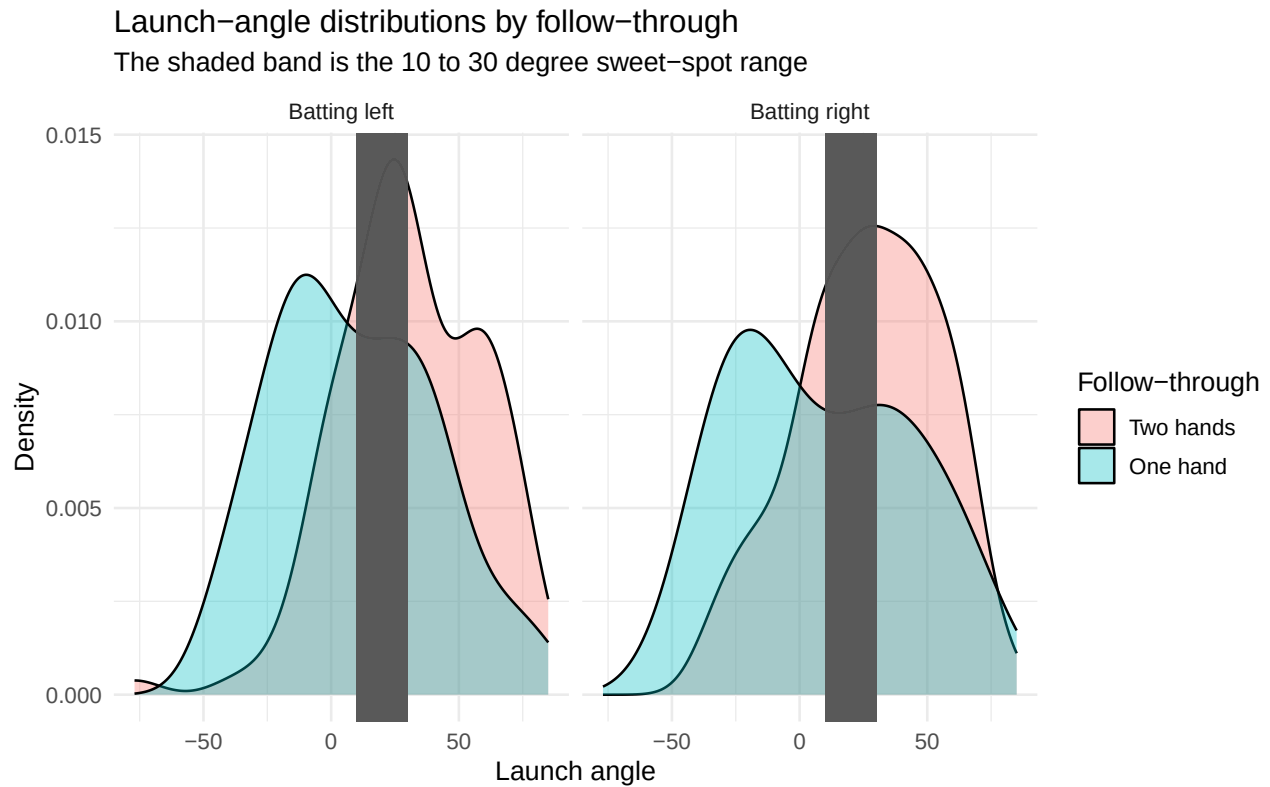
```



```

contact_swings %>%
  ggplot(aes(x = launch_angle, fill = follow_type)) +
  geom_density(alpha = 0.35) +
  geom_rect(
    aes(xmin = 10, xmax = 30, ymin = -Inf, ymax = Inf),
    inherit.aes = FALSE,
    alpha = 0.12
  ) +
  facet_wrap(~ batter_side) +
  labs(
    x = "Launch angle",
    y = "Density",
    fill = "Follow-through",
    title = "Launch-angle distributions by follow-through",
    subtitle = "The shaded band is the 10 to 30 degree sweet-spot range"
  )

```



These plots are descriptive. They do not prove that keeping two hands on the bat causes better contact, because follow-through type may itself be affected by pitch location, count, timing, or swing adjustability. That is why the next section treats follow-through choice as a modeling problem.

Contact Quality On Pitches In The Strike Zone

One possible explanation for the weaker one-hand results is that many one-hand swings happen on pitches outside the strike zone. To check that, the table below repeats the contact-quality comparison using only pitches in Statcast zones 1 through 9.

```
in_zone_contact = contact_swings %>%
  filter(in_zone)

in_zone_contact %>%
  group_by(batter_side, follow_type) %>%
  summarize(
    n = n(),
    avg_ev = mean(launch_speed, na.rm = TRUE),
    median_ev = median(launch_speed, na.rm = TRUE),
    hard_hit_rate = mean(hard_hit, na.rm = TRUE),
    avg_la = mean(launch_angle, na.rm = TRUE),
    sweet_spot_rate = mean(sweet_spot, na.rm = TRUE),
    productive_contact_rate = mean(productive_contact, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  kable(digits = 3, caption = "Batted-ball quality on in-zone pitches only")
```

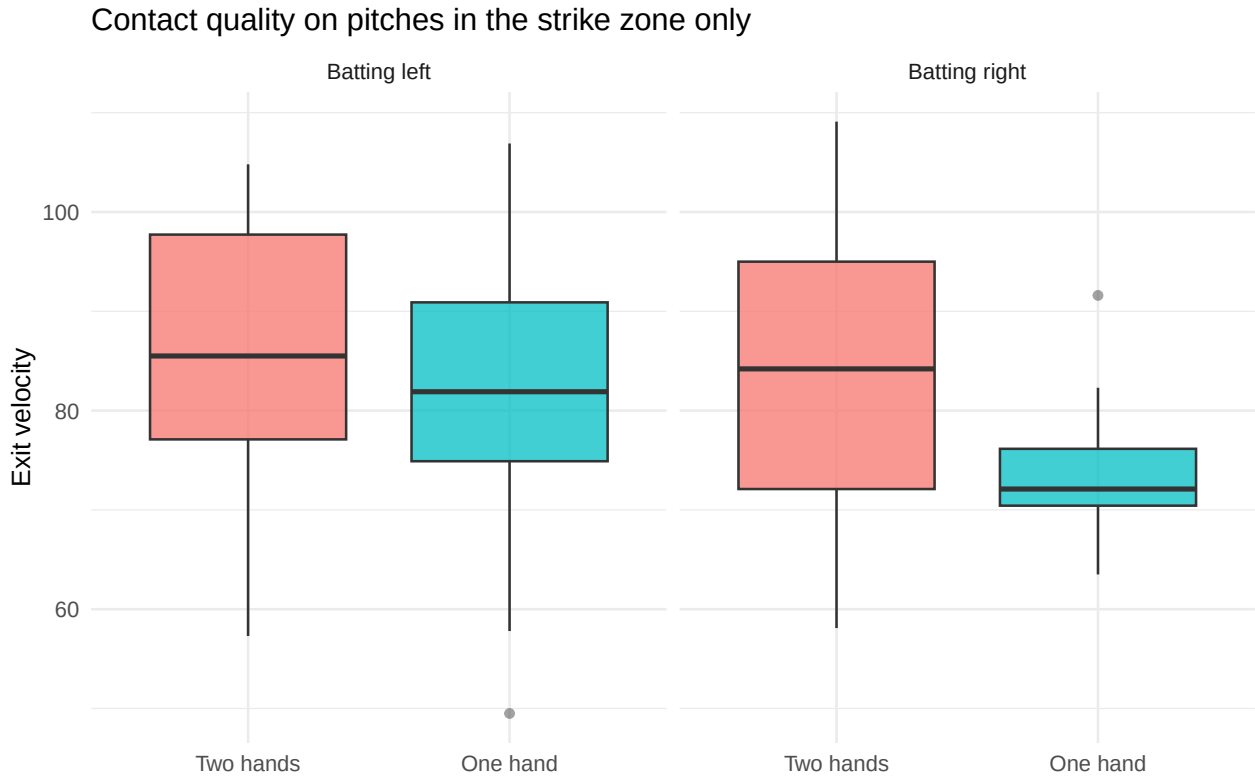
Table 5: Batted-ball quality on in-zone pitches only

batter_side	follow_type	n	avg_ev	median_ev	hard_hit_rate	avg_la	sweet_spot_rate	productive_contact_rate
Batting left	Two hands	92	86.188	85.5	0.315	31.522	0.293	0.163
Batting left	One hand	69	82.101	81.9	0.116	7.942	0.159	0.014
Batting right	Two hands	101	83.985	84.2	0.257	28.455	0.257	0.119
Batting right	One hand	12	73.917	72.1	0.000	14.167	0.167	0.000

```

in_zone_contact %>%
  ggplot(aes(x = follow_type, y = launch_speed, fill = follow_type)) +
  geom_boxplot(alpha = 0.75, outlier.alpha = 0.45) +
  facet_wrap(~ batter_side) +
  labs(
    x = NULL,
    y = "Exit velocity",
    fill = "Follow-through",
    title = "Contact quality on pitches in the strike zone only"
  ) +
  guides(fill = "none")

```



This in-zone restriction is important because it removes the most obvious pitch-location excuse. If the two-hand advantage remains on pitches in the strike zone, then the follow-through difference is not only a chase-pitch artifact.

Alternative Explanations Checked

The original notebook worked through many possible explanations one at a time. This section preserves that logic in the final report. The goal is not to prove causation with any single graph, but to show that the obvious explanations do not fully account for the one-hand versus two-hand difference.

Two-Strike Counts

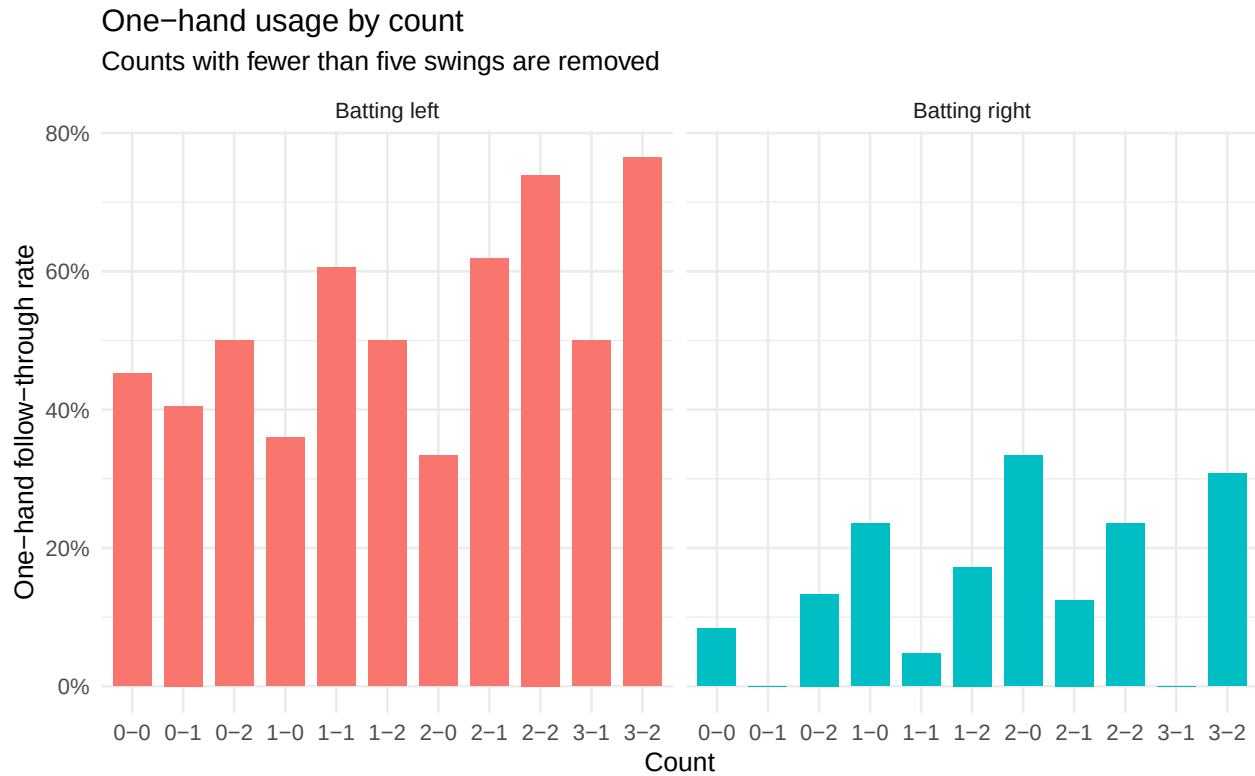
One natural explanation is that Carlson releases one hand because he is protecting the zone with two strikes. If that were the whole story, one-hand swings would be mostly concentrated in two-strike counts.

```
full_swings %>%
  count(batter_side, two_strikes, follow_type) %>%
  group_by(batter_side, two_strikes) %>%
  mutate(rate = n / sum(n)) %>%
  ungroup() %>%
  kable(digits = 3, caption = "Follow-through usage by batting side and two-strike count")
```

Table 6: Follow-through usage by batting side and two-strike count

batter_side	two_strikes	follow_type	n	rate
Batting left	FALSE	Two hands	95	0.519
Batting left	FALSE	One hand	88	0.481
Batting left	TRUE	Two hands	40	0.400
Batting left	TRUE	One hand	60	0.600
Batting right	FALSE	Two hands	92	0.893
Batting right	FALSE	One hand	11	0.107
Batting right	TRUE	Two hands	59	0.797
Batting right	TRUE	One hand	15	0.203

```
full_swings %>%
  group_by(batter_side, count_state) %>%
  summarize(
    n = n(),
    one_hand_rate = mean(one_hand),
    .groups = "drop"
  ) %>%
  filter(n >= 5) %>%
  ggplot(aes(x = count_state, y = one_hand_rate, fill = batter_side)) +
  geom_col(width = 0.75) +
  facet_wrap(~ batter_side) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    x = "Count",
    y = "One-hand follow-through rate",
    fill = "Batting side",
    title = "One-hand usage by count",
    subtitle = "Counts with fewer than five swings are removed"
  ) +
  guides(fill = "none")
```



Two-strike counts matter, but they do not fully explain the pattern. Carlson still uses both follow-throughs across different counts, especially when batting left-handed.

Chase Pitches And Pitch Location

A second explanation is pitch location. One-hand follow-throughs may happen more often when Carlson is reaching for pitches outside the strike zone.

```
full_swings %>%
  group_by(batter_side, follow_type) %>%
  summarize(
    n = n(),
    chase_rate = mean(chase),
    in_zone_rate = mean(in_zone),
    .groups = "drop"
  ) %>%
  kable(digits = 3, caption = "Chase and in-zone rates by follow-through")
```

Table 7: Chase and in-zone rates by follow-through

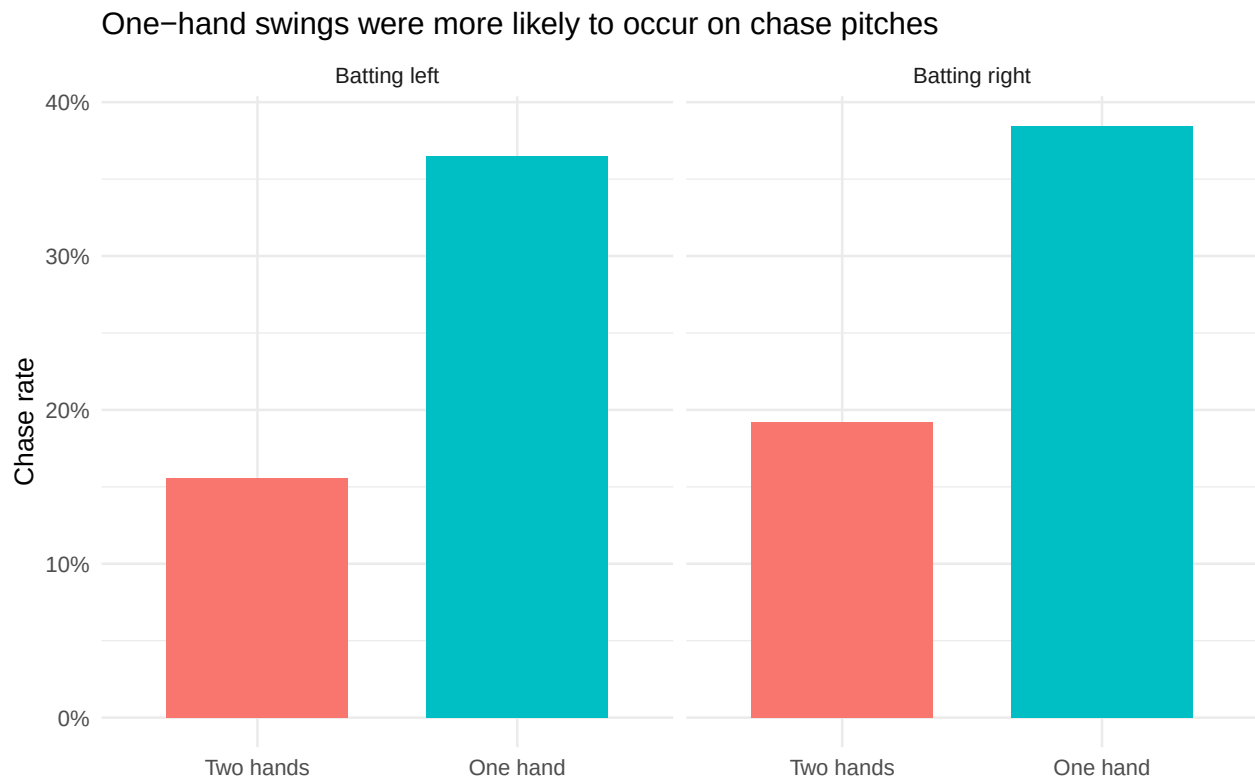
batter_side	follow_type	n	chase_rate	in_zone_rate
Batting left	Two hands	135	0.156	0.844
Batting left	One hand	148	0.365	0.635
Batting right	Two hands	151	0.192	0.808
Batting right	One hand	26	0.385	0.615

```
full_swings %>%
  group_by(batter_side, follow_type) %>%
  summarize(chase_rate = mean(chase), .groups = "drop") %>%
```

```

ggplot(aes(x = follow_type, y = chase_rate, fill = follow_type)) +
  geom_col(width = 0.7) +
  facet_wrap(~ batter_side) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    x = NULL,
    y = "Chase rate",
    fill = "Follow-through",
    title = "One-hand swings were more likely to occur on chase pitches"
  ) +
  guides(fill = "none")

```



```

zone_locations = tibble(
  zone_num = 1:14,
  zone_x = c(1, 2, 3, 1, 2, 3, 1, 2, 3, 0, 4, 0, 4, 2),
  zone_y = c(3, 3, 3, 2, 2, 2, 1, 1, 1, 0, 0, 4, 4, 5)
)

full_swings %>%
  mutate(zone_num = as.integer(as.character(zone))) %>%
  group_by(batter_side, zone_num) %>%
  summarize(
    n = n(),
    one_hand_rate = mean(one_hand),
    .groups = "drop"
  ) %>%
  left_join(zone_locations, by = "zone_num") %>%
  ggplot(aes(x = zone_x, y = zone_y, fill = one_hand_rate)) +
  geom_tile(color = "white", linewidth = 0.8) +

```

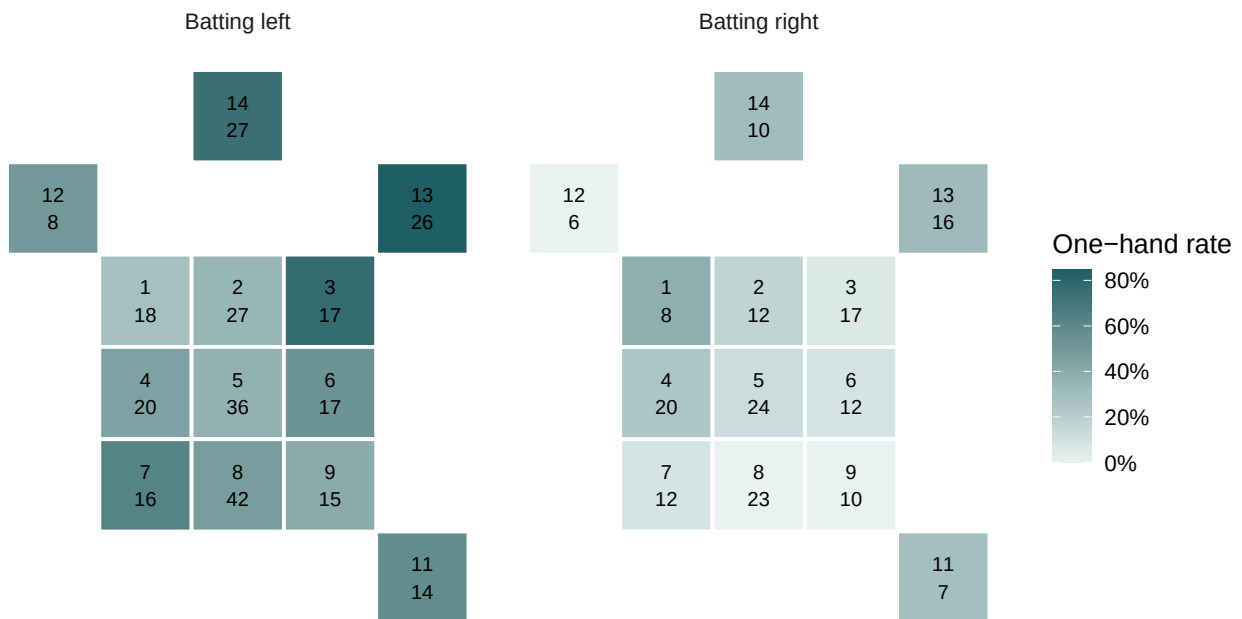
```

geom_text(aes(label = paste0(zone_num, "\n", n)), size = 3) +
facet_wrap(~ batter_side) +
scale_fill_gradient(low = "#e8f3f1", high = "#1e5f63", labels = scales::percent_format()) +
coord_equal() +
labs(
  x = NULL,
  y = NULL,
  fill = "One-hand rate",
  title = "One-hand rate by Statcast zone",
  subtitle = "Tile labels show zone number and swing count"
) +
theme(
  axis.text = element_blank(),
  axis.ticks = element_blank(),
  panel.grid = element_blank()
)

```

One-hand rate by Statcast zone

Tile labels show zone number and swing count



Location clearly matters. One-hand swings are more common outside the zone. However, the in-zone contact-quality section above shows that the contact-quality gap remains even when the analysis is restricted to pitches in the strike zone.

Pitch Velocity

Another possible explanation is pitch speed. If one-hand follow-throughs were mostly a reaction to very slow or very fast pitches, pitch velocity should explain much of the pattern.

```

full_swings %>%
mutate(
  velocity_bucket = case_when(
    release_speed < 85 ~ "Under 85 mph",
    release_speed <= 95 ~ "85 to 95 mph",
    release_speed > 95 ~ "Over 95 mph"
  )
)

```

```

),
  velocity_bucket = factor(
    velocity_bucket,
    levels = c("Under 85 mph", "85 to 95 mph", "Over 95 mph")
  )
) %>%
group_by(batter_side, velocity_bucket, follow_type) %>%
summarize(
  n = n(),
  avg_ev = mean(launch_speed, na.rm = TRUE),
  .groups = "drop"
) %>%
kable(digits = 3, caption = "Follow-through usage and average exit velocity by pitch velocity bucket")

```

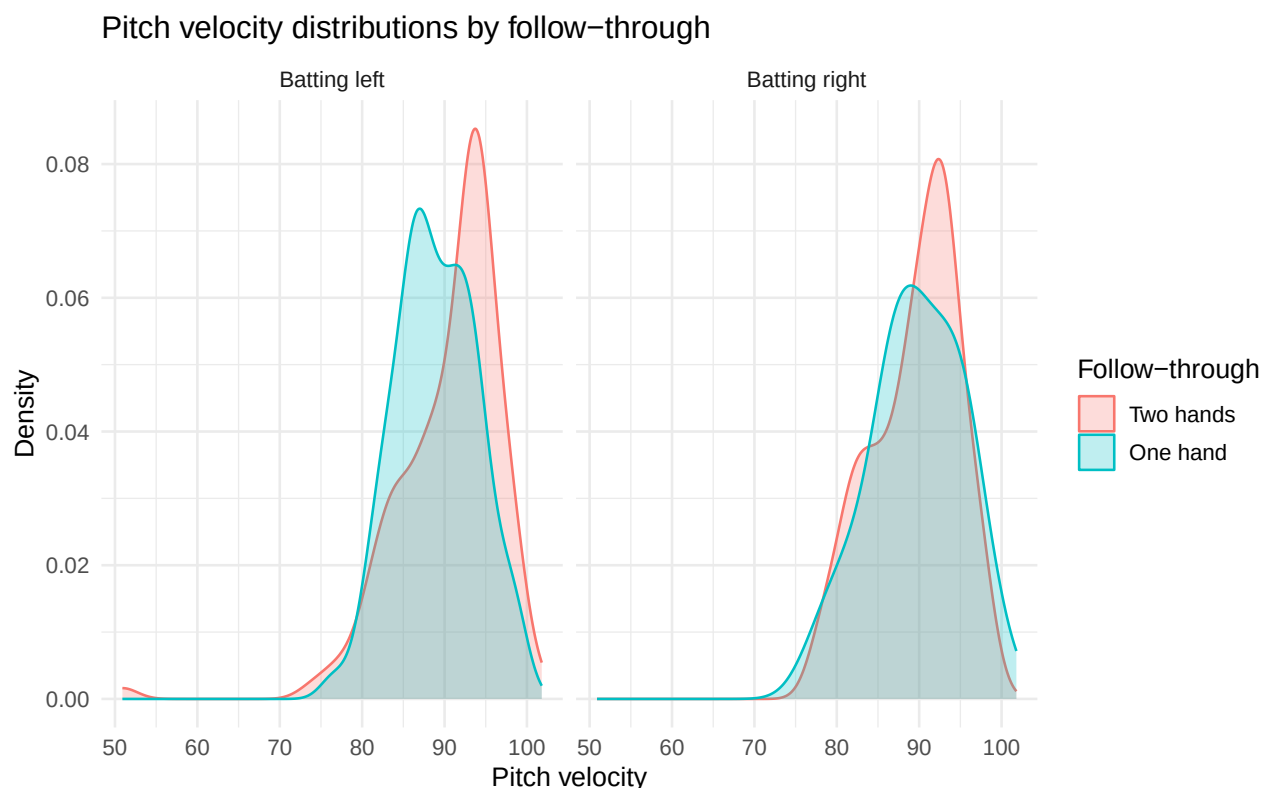
Table 8: Follow-through usage and average exit velocity by pitch velocity bucket

batter_side	velocity_bucket	follow_type	n	avg_ev
Batting left	Under 85 mph	Two hands	25	82.665
Batting left	Under 85 mph	One hand	31	76.867
Batting left	85 to 95 mph	Two hands	83	86.792
Batting left	85 to 95 mph	One hand	102	82.697
Batting left	Over 95 mph	Two hands	27	86.464
Batting left	Over 95 mph	One hand	15	80.409
Batting right	Under 85 mph	Two hands	34	83.021
Batting right	Under 85 mph	One hand	4	67.467
Batting right	85 to 95 mph	Two hands	98	83.239
Batting right	85 to 95 mph	One hand	17	69.718
Batting right	Over 95 mph	Two hands	19	80.062
Batting right	Over 95 mph	One hand	5	75.650

```

full_swings %>%
ggplot(aes(x = release_speed, color = follow_type, fill = follow_type)) +
geom_density(alpha = 0.25) +
facet_wrap(~ batter_side) +
labs(
  x = "Pitch velocity",
  y = "Density",
  color = "Follow-through",
  fill = "Follow-through",
  title = "Pitch velocity distributions by follow-through"
)

```



Pitch velocity does not appear to be the main explanation. The formal models below reach the same conclusion: once batting side, location, count, and other pitch characteristics are included, raw velocity is not one of the strongest predictors of follow-through choice.

Taken together, these checks match the original notebook’s main exploratory conclusion. Count, location, and velocity all provide useful context, but none of them fully explain why Carlson’s one-hand follow-throughs are associated with worse contact quality.

Which Statcast Variables Explain Follow-Through?

The first statistical question is:

Which Statcast variables are most important for predicting whether a full swing ends with one hand?

Here, “important” means important inside the models used in this report. It does not mean causal importance. A variable can be statistically useful for predicting one-hand follow-throughs without being the direct reason Carlson released one hand from the bat.

The candidate predictors include:

- batting side (`stand`)
- pitch type group
- count variables (`balls`, `strikes`)
- strike-zone indicator and exact pitch location (`in_zone`, `plate_x`, `plate_z`)
- pitch velocity and movement (`release_speed`, `pfx_x`, `pfx_z`)
- spin and extension (`release_spin_rate`, `release_extension`)
- game context (`outs_when_up`, `n_thruorder_pitcher`)

I do not include batted-ball result variables such as launch speed or launch angle in this follow-through-choice model. The reason is timing: those variables are outcomes of the swing after contact occurs. They are useful

for answering whether one follow-through produced better contact, but they should not be used to answer why Carlson ended up using one follow-through instead of the other.

```
# Keep only the variables used in the follow-through model.
model_df = full_swings %>%
  select(
    one_hand, follow_type, batter_side, pitch_group,
    balls, strikes, in_zone, release_speed,
    plate_x, plate_z, pfx_x, pfx_z,
    release_spin_rate, release_extension,
    outs_when_up, n_thruorder_pitcher
  ) %>%
  drop_na()

# These numeric variables are standardized before modeling.
# For example, release_speed_z is the standardized version of release_speed.
numeric_vars = c(
  "balls", "strikes", "release_speed", "plate_x", "plate_z",
  "pfx_x", "pfx_z", "release_spin_rate", "release_extension",
  "outs_when_up", "n_thruorder_pitcher"
)

model_df_z = model_df %>%
  mutate(across(all_of(numeric_vars), ~ as.numeric(scale(.x)), .names = "{.col}_z"))

follow_formula = one_hand ~ batter_side + pitch_group + balls_z + strikes_z +
  in_zone + release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
  release_spin_rate_z + release_extension_z + outs_when_up_z +
  n_thruorder_pitcher_z

follow_model = glm(follow_formula, data = model_df_z, family = binomial())

glance(follow_model) %>%
  select(null.deviance, df.null, deviance, df.residual, AIC, BIC) %>%
  kable(digits = 3, caption = "Logistic model summary for one-hand follow-through")
```

Table 9: Logistic model summary for one-hand follow-through

null.deviance	df.null	deviance	df.residual	AIC	BIC
600.548	451	480.479	436	512.479	578.298

Drop-One Importance

The table below removes each predictor from the model one at a time. A large likelihood-ratio statistic or large increase in AIC means the variable contributed more information after accounting for the other variables.

```
drop_one = drop1(follow_model, test = "Chisq") %>%
  as.data.frame() %>%
  rownames_to_column("term") %>%
  filter(term != "<none>") %>%
  mutate(
    delta_AIC = AIC - AIC(follow_model),
    p.value = `Pr(>Chi)`
```

```

) %>%
  arrange(desc(LRT))

drop_one %>%
  select(term, Df, LRT, p.value, delta_AIC) %>%
  kable(digits = 4, caption = "Drop-one likelihood-ratio importance for follow-through choice")

```

Table 10: Drop-one likelihood-ratio importance for follow-through choice

term	Df	LRT	p.value	delta_AIC
batter_side	1	54.8430	0.0000	52.8430
in_zone	1	12.9662	0.0003	10.9662
balls_z	1	6.9865	0.0082	4.9865
plate_z_z	1	3.8777	0.0489	1.8777
pitch_group	2	3.0978	0.2125	-0.9022
pfx_z_z	1	2.0484	0.1524	0.0484
strikes_z	1	1.8352	0.1755	-0.1648
plate_x_z	1	0.9279	0.3354	-1.0721
release_extension_z	1	0.6399	0.4237	-1.3601
release_speed_z	1	0.3180	0.5728	-1.6820
pfx_x_z	1	0.2532	0.6149	-1.7468
n_thruorder_pitcher_z	1	0.0965	0.7561	-1.9035
release_spin_rate_z	1	0.0356	0.8503	-1.9644
outs_when_up_z	1	0.0095	0.9224	-1.9905

Coefficient View

Because the numeric variables were standardized before modeling, their coefficients are roughly comparable. Positive coefficients increase the estimated probability of a one-hand follow-through; negative coefficients decrease it.

```

coef_tbl = tidy(follow_model, conf.int = TRUE) %>%
  filter(term != "(Intercept)") %>%
  mutate(
    term = str_replace_all(term, "_z", ""),
    odds_ratio = exp(estimate),
    conf.low.or = exp(conf.low),
    conf.high.or = exp(conf.high)
  ) %>%
  arrange(desc(abs(estimate)))

coef_tbl %>%
  select(term, estimate, std.error, statistic, p.value, odds_ratio) %>%
  kable(digits = 4, caption = "Standardized logistic coefficients for one-hand follow-through")

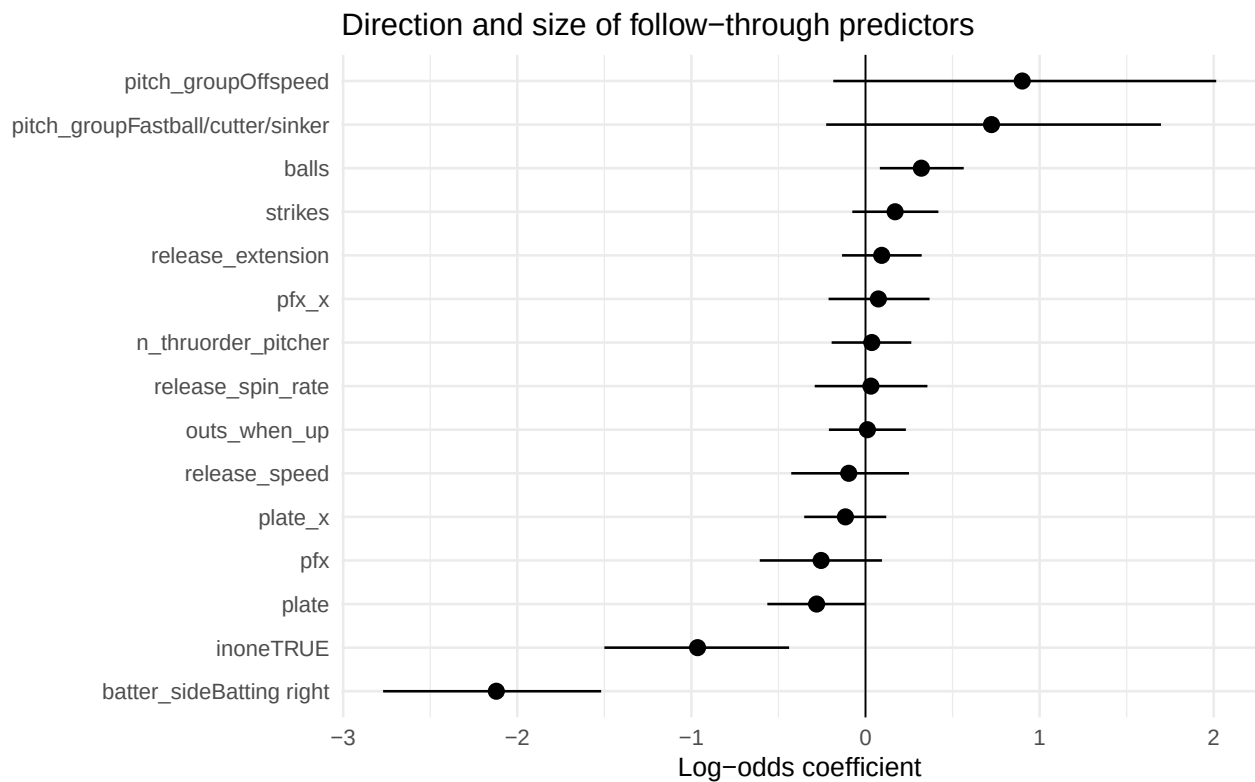
```

Table 11: Standardized logistic coefficients for one-hand follow-through

term	estimate	std.error	statistic	p.value	odds_ratio
batter_sideBatting right	-2.1205	0.3183	-6.6617	0.0000	0.1200
inoneTRUE	-0.9646	0.2700	-3.5729	0.0004	0.3812

term	estimate	std.error	statistic	p.value	odds_ratio
pitch_groupOffspeed	0.9002	0.5595	1.6089	0.1076	2.4602
pitch_groupFastball/cutter/sinker	0.7237	0.4890	1.4800	0.1389	2.0621
balls	0.3208	0.1225	2.6178	0.0088	1.3782
plate	-0.2807	0.1431	-1.9617	0.0498	0.7552
pfx	-0.2550	0.1784	-1.4300	0.1527	0.7749
strikes	0.1700	0.1259	1.3505	0.1768	1.1853
plate_x	-0.1153	0.1199	-0.9617	0.3362	0.8911
release_speed	-0.0962	0.1688	-0.5697	0.5689	0.9083
release_extension	0.0930	0.1164	0.7989	0.4243	1.0975
pfx_x	0.0739	0.1474	0.5015	0.6160	1.0767
n_thruorder_pitcher	0.0362	0.1164	0.3111	0.7558	1.0369
release_spin_rate	0.0311	0.1647	0.1887	0.8503	1.0316
outs_when_up	0.0110	0.1126	0.0974	0.9224	1.0110

```
coef_tbl %>%
  mutate(term = fct_reorder(term, estimate)) %>%
  ggplot(aes(x = term, y = estimate)) +
  geom_hline(yintercept = 0, linewidth = 0.4) +
  geom_pointrange(aes(ymin = conf.low, ymax = conf.high)) +
  coord_flip() +
  labs(
    x = NULL,
    y = "Log-odds coefficient",
    title = "Direction and size of follow-through predictors"
  )
```



Cross-Validated Permutation Importance

Likelihood-ratio tests are model-based. As a second view, I estimate out-of-sample predictive importance using five-fold cross-validation and permutation importance. For each validation fold, I fit the logistic model on the other folds, calculate baseline log loss, then shuffle one predictor in the validation fold. If shuffling a variable makes log loss worse, that variable was useful for prediction.

```
# This function measures prediction error for a yes/no outcome.
# Smaller log loss means better predictions.
log_loss = function(actual, predicted_probability) {
  predicted_probability = pmin(pmax(predicted_probability, 1e-15), 1 - 1e-15)
  -mean(
    actual * log(predicted_probability) +
    (1 - actual) * log(1 - predicted_probability)
  )
}
```

```
perm_vars = c(
  "batter_side", "pitch_group", "balls_z", "strikes_z", "in_zone",
  "release_speed_z", "plate_x_z", "plate_z_z", "pfx_x_z", "pfx_z_z",
  "release_spin_rate_z", "release_extension_z", "outs_when_up_z",
  "n_thruorder_pitcher_z"
)

# Create five folds for cross-validation.
model_df_z$fold_id = sample(rep(1:5, length.out = nrow(model_df_z)))

perm_results = list()
result_number = 1

for (fold in 1:5) {
  train_data = model_df_z %>%
    filter(fold_id != fold)

  validation_data = model_df_z %>%
    filter(fold_id == fold)

  fold_model = glm(follow_formula, data = train_data, family = binomial())

  baseline_predictions = predict(
    fold_model,
    newdata = validation_data,
    type = "response"
  )

  baseline_loss = log_loss(validation_data$one_hand, baseline_predictions)

  for (variable_name in perm_vars) {
    shuffled_data = validation_data
    shuffled_data[[variable_name]] = sample(shuffled_data[[variable_name]])

    shuffled_predictions = predict(
      fold_model,
      newdata = shuffled_data,
      type = "response"
    )
  }
}
```

```

)

shuffled_loss = log_loss(validation_data$one_hand, shuffled_predictions)

perm_results[[result_number]] = tibble(
  fold = fold,
  variable = variable_name,
  baseline_log_loss = baseline_loss,
  permuted_log_loss = shuffled_loss,
  loss_increase = shuffled_loss - baseline_loss
)

result_number = result_number + 1
}
}

perm_importance = bind_rows(perm_results)

perm_summary = perm_importance %>%
  group_by(variable) %>%
  summarize(
    mean_loss_increase = mean(loss_increase),
    sd_loss_increase = sd(loss_increase),
    .groups = "drop"
  ) %>%
  mutate(variable = str_replace_all(variable, "_z", "")) %>%
  arrange(desc(mean_loss_increase))

perm_summary %>%
  kable(digits = 4, caption = "Cross-validated permutation importance")

```

Table 12: Cross-validated permutation importance

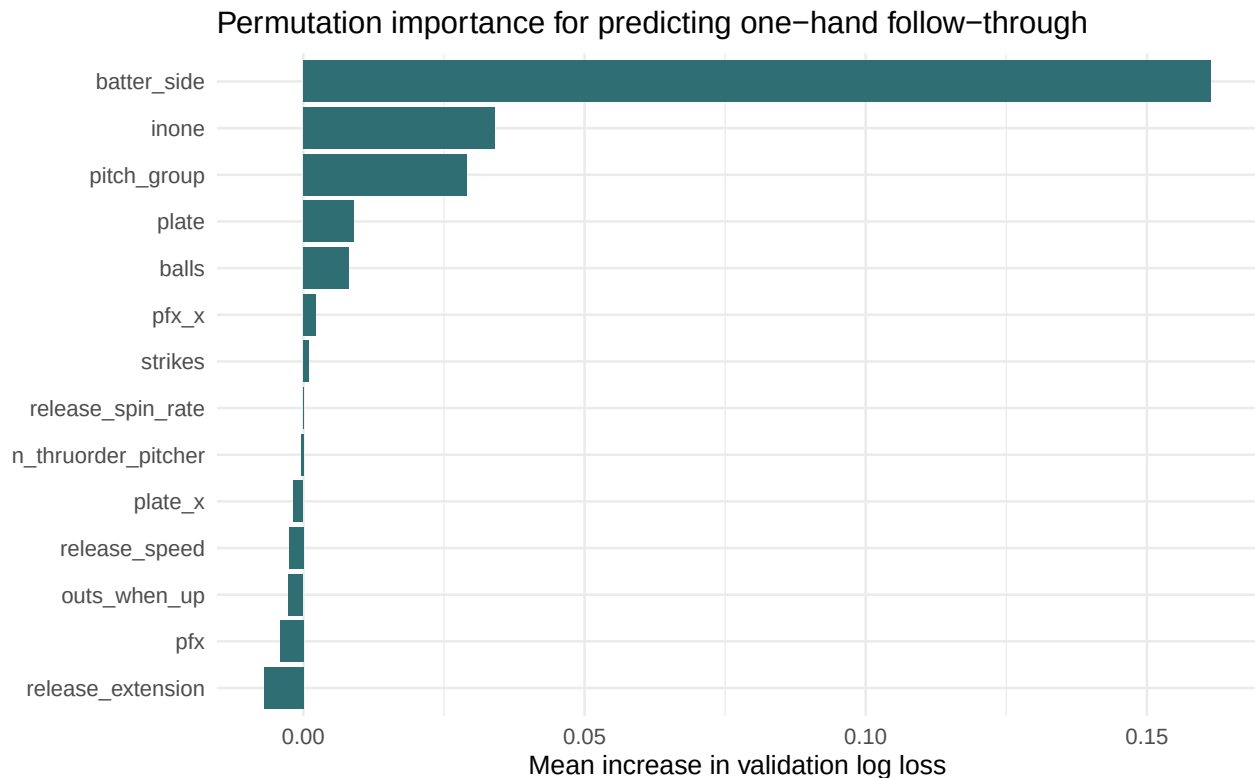
variable	mean_loss_increase	sd_loss_increase
batter_side	0.1613	0.0576
inone	0.0341	0.0284
pitch_group	0.0292	0.0140
plate	0.0090	0.0251
balls	0.0080	0.0291
pfx_x	0.0023	0.0068
strikes	0.0009	0.0096
release_spin_rate	0.0001	0.0031
n_thruorder_pitcher	-0.0004	0.0020
plate_x	-0.0018	0.0061
release_speed	-0.0025	0.0131
outs_when_up	-0.0027	0.0036
pfx	-0.0042	0.0123
release_extension	-0.0070	0.0154

```

perm_summary %>%
  mutate(variable = fct_reorder(variable, mean_loss_increase)) %>%
  ggplot(aes(x = variable, y = mean_loss_increase)) +

```

```
geom_col(fill = "#2f6f73") +
coord_flip() +
labs(
  x = NULL,
  y = "Mean increase in validation log loss",
  title = "Permutation importance for predicting one-hand follow-through"
)
```



Interpretation

The most important variable is expected to be batting side: Carlson's right-handed swing was much more consistently two-handed. After that, the important variables are the ones that describe the pitch situation. In particular, pitches outside the strike zone and count variables are important because one-hand follow-throughs often occur when Carlson is reacting to a less hittable pitch rather than taking his cleanest swing.

Variables that may sound important in theory, such as raw pitch velocity, are less important in this one-season sample once pitch location, count, and batting side are included. That does not mean velocity never matters; it means this data set does not provide strong evidence that velocity is one of the main drivers of Carlson's follow-through choice.

Does Follow-Through Still Matter After Statcast Context?

The previous model asks what predicts one-hand follow-through. The next question is whether follow-through remains associated with contact quality after adjusting for Statcast context.

```
contact_model_df = full_swings %>%
  filter(!is.na(launch_speed), !is.na(launch_angle)) %>%
  select(
    launch_speed, launch_angle, hard_hit, sweet_spot, productive_contact,
```

```

follow_type, batter_side, pitch_group, balls, strikes, in_zone,
release_speed, plate_x, plate_z, pfx_x, pfx_z,
release_spin_rate, release_extension, outs_when_up, n_thruorder_pitcher
) %>%
drop_na() %>%
# Recreate standardized versions of the numeric variables for this smaller
# contact-only data set.
mutate(across(all_of(numeric_vars), ~ as.numeric(scale(.x)), .names = "{.col}_z"))

ev_formula = launch_speed ~ follow_type + batter_side + pitch_group +
balls_z + strikes_z + in_zone + release_speed_z + plate_x_z + plate_z_z +
pfx_x_z + pfx_z_z + release_spin_rate_z + release_extension_z +
outs_when_up_z + n_thruorder_pitcher_z

ev_model = lm(ev_formula, data = contact_model_df)

tidy(ev_model, conf.int = TRUE) %>%
  filter(term != "(Intercept)") %>%
  arrange(p.value) %>%
  select(term, estimate, conf.low, conf.high, p.value) %>%
  kable(digits = 4, caption = "Linear model for exit velocity")

```

Table 13: Linear model for exit velocity

term	estimate	conf.low	conf.high	p.value
batter_sideBatting right	-6.3187	-9.7678	-2.8696	0.0004
follow_typeOne hand	-5.4188	-8.5118	-2.3258	0.0006
release_spin_rate_z	-2.0823	-4.0209	-0.1437	0.0354
in_zoneTRUE	3.6654	0.2255	7.1053	0.0368
balls_z	1.2717	-0.1783	2.7217	0.0854
plate_x_z	0.9247	-0.4807	2.3302	0.1964
release_extension_z	-0.7978	-2.1846	0.5889	0.2585
n_thruorder_pitcher_z	0.7046	-0.6397	2.0488	0.3032
pfx_z_z	1.1298	-1.0382	3.2978	0.3060
pfx_x_z	0.8150	-0.7897	2.4197	0.3184
strikes_z	-0.7356	-2.2144	0.7433	0.3285
plate_z_z	-0.6938	-2.3129	0.9253	0.3998
pitch_groupFastball/cutter/sinker	2.1743	-3.8484	8.1969	0.4781
outs_when_up_z	0.3946	-0.9700	1.7593	0.5698
pitch_groupOffspeed	-0.9025	-7.8968	6.0918	0.7998
release_speed_z	-0.0934	-2.1591	1.9723	0.9292

```

hard_hit_formula = hard_hit ~ follow_type + batter_side + pitch_group +
balls_z + strikes_z + in_zone + release_speed_z + plate_x_z + plate_z_z +
pfx_x_z + pfx_z_z + release_spin_rate_z + release_extension_z +
outs_when_up_z + n_thruorder_pitcher_z

hard_hit_model = glm(hard_hit_formula, data = contact_model_df, family = binomial())

drop1(hard_hit_model, test = "Chisq") %>%
  as.data.frame() %>%
  rownames_to_column("term") %>%

```

```

filter(term != "<none>") %>%
mutate(delta_AIC = AIC - AIC(hard_hit_model)) %>%
arrange(desc(LRT)) %>%
select(term, Df, LRT, `Pr(>Chi)`, delta_AIC) %>%
kable(digits = 4, caption = "Drop-one importance for hard-hit probability")

```

Table 14: Drop-one importance for hard-hit probability

term	Df	LRT	Pr(>Chi)	delta_AIC
follow_type	1	15.6226	0.0001	13.6226
batter_side	1	3.2397	0.0719	1.2397
plate_x_z	1	2.2202	0.1362	0.2202
pitch_group	2	1.9304	0.3809	-2.0696
balls_z	1	1.8071	0.1789	-0.1929
release_speed_z	1	1.4043	0.2360	-0.5957
plate_z_z	1	1.3249	0.2497	-0.6751
in_zone	1	1.2055	0.2722	-0.7945
release_extension_z	1	1.0463	0.3064	-0.9537
n_thruorder_pitcher_z	1	1.0004	0.3172	-0.9996
outs_when_up_z	1	0.6838	0.4083	-1.3162
release_spin_rate_z	1	0.1130	0.7367	-1.8870
pfx_z_z	1	0.0515	0.8205	-1.9485
strikes_z	1	0.0199	0.8878	-1.9801
pfx_x_z	1	0.0039	0.9499	-1.9961

```

productive_formula = productive_contact ~ follow_type + batter_side + pitch_group +
  balls_z + strikes_z + in_zone + release_speed_z + plate_x_z + plate_z_z +
  pfx_x_z + pfx_z_z + release_spin_rate_z + release_extension_z +
  outs_when_up_z + n_thruorder_pitcher_z

productive_model = glm(productive_formula, data = contact_model_df, family = binomial())

tidy(productive_model, conf.int = TRUE) %>%
  filter(term != "(Intercept)") %>%
  mutate(odds_ratio = exp(estimate)) %>%
  arrange(p.value) %>%
  select(term, estimate, odds_ratio, conf.low, conf.high, p.value) %>%
  kable(digits = 4, caption = "Logistic model for productive contact")

```

Table 15: Logistic model for productive contact

term	estimate	odds_ratio	conf.low	conf.high	p.value
follow_typeOne hand	-1.7216	0.1788	-3.1011	-0.5875	0.0062
plate_x_z	0.4743	1.6069	-0.0135	0.9904	0.0622
pfx_x_z	-0.4559	0.6339	-0.9903	0.0494	0.0828
in_zoneTRUE	1.1663	3.2102	-0.0935	2.7814	0.1036
release_speed_z	-0.4145	0.6606	-1.0367	0.1166	0.1342
pfx_z_z	0.4286	1.5352	-0.1706	1.0755	0.1731
plate_z_z	-0.3372	0.7137	-0.8874	0.1868	0.2151
outs_when_up_z	-0.2438	0.7836	-0.6692	0.1647	0.2480
strikes_z	0.2558	1.2915	-0.2105	0.7454	0.2906

term	estimate	odds_ratio	conf.low	conf.high	p.value
release_spin_rate_z	0.2847	1.3294	-0.3056	0.8952	0.3500
pitch_groupOffspeed	0.8421	2.3212	-1.1740	2.8534	0.4075
release_extension_z	0.1690	1.1842	-0.2628	0.6172	0.4490
batter_sideBatting right	-0.3740	0.6880	-1.4294	0.6599	0.4790
n_thruorder_pitcher_z	0.1099	1.1162	-0.3039	0.4898	0.5826
balls_z	0.0313	1.0318	-0.4381	0.4819	0.8932
pitch_groupFastball/cutter/sinker	-0.0085	0.9916	-1.7846	1.7994	0.9926

This section should be interpreted carefully because batted-ball models only use swings where contact information is available. Misses and fouls are not represented in launch-speed and launch-angle models the same way as balls in play. Still, these models are useful because they test whether the follow-through signal survives after including the Statcast variables that describe pitch context.

Do The Context Variables Explain Away The Difference?

In the original notebook, I looked through many combinations of handedness, count, pitch location, pitch velocity, and contact quality to see if one variable could explain the difference between one-hand and two-hand follow-throughs. The statistical version of that same idea is a nested-model comparison.

For each contact-quality outcome, I fit two models:

1. a context-only model using batting side, pitch group, count, location, velocity, movement, spin, extension, outs, and pitcher time through the order
2. the same model with follow-through type added

If follow-through adds meaningful information after all of those context variables are included, then the observed difference is not fully explained by the Statcast context variables.

```
# These are the context variables used to explain contact quality.
# The first model excludes follow-through.
# The second model adds follow-through.

ev_context_model = lm(
  launch_speed ~ batter_side + pitch_group + balls_z + strikes_z + in_zone +
    release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = contact_model_df
)

ev_follow_model = lm(
  launch_speed ~ follow_type + batter_side + pitch_group + balls_z + strikes_z +
    in_zone + release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = contact_model_df
)

hard_hit_context_model = glm(
  hard_hit ~ batter_side + pitch_group + balls_z + strikes_z + in_zone +
    release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = contact_model_df,
```

```

    family = binomial()
  )

hard_hit_follow_model = glm(
  hard_hit ~ follow_type + batter_side + pitch_group + balls_z + strikes_z +
    in_zone + release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = contact_model_df,
  family = binomial()
)

productive_context_model = glm(
  productive_contact ~ batter_side + pitch_group + balls_z + strikes_z + in_zone +
    release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = contact_model_df,
  family = binomial()
)

productive_follow_model = glm(
  productive_contact ~ follow_type + batter_side + pitch_group + balls_z +
    strikes_z + in_zone + release_speed_z + plate_x_z + plate_z_z + pfx_x_z +
    pfx_z_z + release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = contact_model_df,
  family = binomial()
)

ev_lrt = anova(ev_context_model, ev_follow_model)
hard_hit_lrt = anova(hard_hit_context_model, hard_hit_follow_model, test = "Chisq")
productive_lrt = anova(productive_context_model, productive_follow_model, test = "Chisq")

nested_results = tibble(
  outcome = c("Exit velocity", "Hard-hit probability", "Productive-contact probability"),
  follow_effect = c(
    coef(ev_follow_model)["follow_typeOne hand"],
    coef(hard_hit_follow_model)["follow_typeOne hand"],
    coef(productive_follow_model)["follow_typeOne hand"]
  ),
  effect_scale = c("mph", "log odds", "log odds"),
  p_value_for_added_follow_through = c(
    ev_lrt$`Pr(>F)`[2],
    hard_hit_lrt$`Pr(>Chi)`[2],
    productive_lrt$`Pr(>Chi)`[2]
  ),
  context_only_AIC = c(
    AIC(ev_context_model),
    AIC(hard_hit_context_model),
    AIC(productive_context_model)
  ),
  with_follow_AIC = c(

```

```

    AIC(ev_follow_model),
    AIC(hard_hit_follow_model),
    AIC(productive_follow_model)
  )
) %>%
  mutate(delta_AIC = with_follow_AIC - context_only_AIC)

nested_results %>%
  kable(digits = 4, caption = "Nested model tests: does follow-through add information after Statcast context?")

```

Table 16: Nested model tests: does follow-through add information after Statcast context?

outcome	follow_effect	effect_scale	value_for_added_follow	context_only_AIC	AIC_follow	delta_AIC
Exit velocity	-5.4188	mph	6e-04	2701.5053	2691.2251	-10.2802
Hard-hit probability	-1.4057	log odds	1e-04	364.9346	351.3120	-13.6226
Productive-contact probability	-1.7216	log odds	2e-03	216.2290	208.7225	-7.5064

The overall model is useful as a first pass, but Carlson is a switch hitter and the original baseball question is really about whether the pattern holds within each side of the plate. The same nested-model comparison can be run separately for left-handed and right-handed swings.

```

stratified_nested_tests = function(data, side_name) {
  side_data = data %>%
    filter(batter_side == side_name) %>%
    droplevels()

  ev_context = lm(
    launch_speed ~ pitch_group + balls_z + strikes_z + in_zone +
      release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
      release_spin_rate_z + release_extension_z + outs_when_up_z +
      n_thruorder_pitcher_z,
    data = side_data
  )

  ev_follow = lm(
    launch_speed ~ follow_type + pitch_group + balls_z + strikes_z + in_zone +
      release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
      release_spin_rate_z + release_extension_z + outs_when_up_z +
      n_thruorder_pitcher_z,
    data = side_data
  )

  ev_test = anova(ev_context, ev_follow)

  hard_context = glm(
    hard_hit ~ pitch_group + balls_z + strikes_z + in_zone +
      release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
      release_spin_rate_z + release_extension_z + outs_when_up_z +
      n_thruorder_pitcher_z,

```

```

data = side_data,
family = binomial()
)

hard_follow = glm(
  hard_hit ~ follow_type + pitch_group + balls_z + strikes_z + in_zone +
    release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = side_data,
  family = binomial()
)

hard_test = anova(hard_context, hard_follow, test = "Chisq")

productive_context = glm(
  productive_contact ~ pitch_group + balls_z + strikes_z + in_zone +
    release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = side_data,
  family = binomial()
)

productive_follow = glm(
  productive_contact ~ follow_type + pitch_group + balls_z + strikes_z +
    in_zone + release_speed_z + plate_x_z + plate_z_z + pfx_x_z + pfx_z_z +
    release_spin_rate_z + release_extension_z + outs_when_up_z +
    n_thruorder_pitcher_z,
  data = side_data,
  family = binomial()
)

productive_test = anova(productive_context, productive_follow, test = "Chisq")

tibble(
  batter_side = side_name,
  outcome = c("Exit velocity", "Hard-hit probability", "Productive-contact probability"),
  n = nrow(side_data),
  one_hand_n = sum(side_data$follow_type == "One hand"),
  two_hand_n = sum(side_data$follow_type == "Two hands"),
  follow_effect = c(
    coef(ev_follow)["follow_typeOne hand"],
    coef(hard_follow)["follow_typeOne hand"],
    coef(productive_follow)["follow_typeOne hand"]
  ),
  effect_scale = c("mph", "log odds", "log odds"),
  p_value_for_added_follow_through = c(
    ev_test$`Pr(>F)`[2],
    hard_test$`Pr(>Chi)`[2],
    productive_test$`Pr(>Chi)`[2]
  ),
  delta_AIC = c(

```

```

    AIC(ev_follow) - AIC(ev_context),
    AIC(hard_follow) - AIC(hard_context),
    AIC(productive_follow) - AIC(productive_context)
  )
)
}

bind_rows(
  stratified_nested_tests(contact_model_df, "Batting left"),
  stratified_nested_tests(contact_model_df, "Batting right")
) %>%
  kable(digits = 4, caption = "Nested model tests separated by batting side")

```

Table 17: Nested model tests separated by batting side

batter_side	outcome	n	one hand	two hand	follow effect	effect scale	p value	for added follow-through	AIC diff
Batting left	Exit velocity	208	102	106	-4.3074	mph	0.0163	-	4.2689
Batting left	Hard-hit probability	208	102	106	-1.3674	log odds	0.0007	-	9.5216
Batting left	Productive-contact probability	208	102	106	-1.5648	log odds	0.0106	-	4.5253
Batting right	Exit velocity	132	18	114	-8.0977	mph	0.0361	-	3.0192
Batting right	Hard-hit probability	132	18	114	-16.7627	log odds	0.0295	-	2.7400
Batting right	Productive-contact probability	132	18	114	-16.4071	log odds	0.4126	1.3288	

For the left-handed swings, where the one-hand and two-hand samples are reasonably balanced, adding follow-through improves all three contact-quality models. One-hand follow-through remains associated with lower exit velocity, lower hard-hit probability, and lower productive-contact probability after the Statcast/context variables are included.

The right-handed split points in the same general direction for exit velocity and hard-hit probability, but the one-hand sample is very small. That makes the right-handed estimates unstable. For this reason, the cleaner statistical evidence is the left-handed result.

For all three contact-quality outcomes in the left-handed sample, adding follow-through improves the model after the Statcast/context variables are already included. The one-hand coefficient remains negative for exit velocity, hard-hit probability, and productive-contact probability. This supports the main finding from the original exploratory notebook: the difference between follow-through types cannot be fully explained by the obvious contextual variables I tested.

That still does not prove causation. The better interpretation is that one-hand follow-through is a strong marker of worse contact quality in this sample, even after accounting for the pitch and count context available in Statcast.

Image-Based Machine Learning Attempt

I also attempted to expand the project beyond Carlson by automating follow-through labels from video frames. The classification target was:

- one-hand follow-through

- two-hand follow-through

This was motivated by the main bottleneck in the project: manual video labeling is slow, and league-wide follow-through analysis would require many thousands of labels. The machine learning goal was therefore not separate from the statistical analysis, but it should be treated as exploratory. It is included to document the pipeline and the lessons learned, not to make a substantive claim about Carlson or about follow-throughs across MLB.

Video Linking and Frame Extraction

The project began with pitch-level Statcast data for Carlson’s 2023 season. Each row included identifiers such as `game_pk`, `at_bat_number`, and `pitch_number`, along with batted-ball metrics such as exit velocity and launch angle. The key engineering step was linking those rows to the actual swing videos.

Using the MLB Stats API endpoint

`https://statsapi.mlb.com/api/v1.1/game/{gamePk}/feed/live`

I matched each Statcast row to a unique `playId`. That `playId` made it possible to construct Baseball Savant video URLs of the form

`https://baseballsavant.mlb.com/sporty-videos?playId=...`

This was the bridge from tabular data to video data: Statcast row to MLB `playId` to Baseball Savant video.

After the video URL was available, I used `yt-dlp` to resolve the direct video stream and `ffmpeg` to extract frames. The extraction window was intentionally short:

- start time: about 3.6 seconds
- duration: about 0.6 seconds
- frame rate: 8 frames per second

This produced roughly five frames per swing, centered around the swing and follow-through phase. Frames were saved with filenames such as `{playId}_001.jpg`, `{playId}_002.jpg`, and so on.

Dataset Construction

The image data set was built from the manually curated **Hands on Swing** labels. The original class folders were:

- `one_hand/`
- `two_hand/`
- `other/`

The **other** class included swings that were not clean one-hand or two-hand follow-through examples, such as check swings or other unusual labels. Later, I simplified the task to a binary one-hand versus two-hand classifier because the three-class problem added ambiguity without helping the core research question.

Data alignment was a major part of the work. Several problems had to be solved:

- missing or misaligned swings
- multiple swings within the same game not always appearing in a simple order
- matching Statcast rows to manually labeled rows
- incorrect manual labels found during re-review

The final solution was to manually verify and add `playId` to each labeled swing, then join the labels to Statcast through that identifier. This is also what makes the statistical analysis in this report reproducible from `carlson_swings_labeled.csv`.

The folder contains the main pieces of that attempt:

- `python scripts/get_video_url.py`: used MLB game feeds to recover `playIds` for Statcast pitches.

- `python scripts/batch_extract_frames.py`: used Baseball Savant video URLs, `yt-dlp`, and `ffmpeg` to extract frames around each swing.
- `python scripts/build_dataset.py`: mapped manual labels to image folders.
- `python scripts/split_dataset.py`: split images into train/validation/test sets by `playId`, so frames from the same swing would not leak across splits.
- `swing_dataset.py`: grouped five frames from the same swing into one 15-channel tensor.
- `train_model.py`: fine-tuned a ResNet18 model to classify one-hand versus two-hand follow-through.
- `test_model.py`, `swing_vote_test.py`, and `swing_avg_test.py`: tried different swing-level evaluation strategies.

```
count_jpgs = function(path) {
  if (!dir.exists(path)) return(tibble(path = path, n_jpg = 0L))
  tibble(
    path = list.dirs(path, recursive = TRUE, full.names = TRUE),
  ) %>%
    mutate(
      n_jpg = map_int(path, ~ length(list.files(.x, pattern = "\\.[jpg$]", ignore.case = TRUE)))
    ) %>%
    filter(n_jpg > 0)
}

bind_rows(
  count_jpgs("dataset") %>% mutate(source = "Full labeled frame dataset"),
  count_jpgs("dataset_split") %>% mutate(source = "Three-class split"),
  count_jpgs("dataset_binary") %>% mutate(source = "Binary split")
) %>%
  select(source, path, n_jpg) %>%
  arrange(source, path) %>%
  kable(caption = "Frame counts from the image classification attempt")
```

Table 18: Frame counts from the image classification attempt

source	path	n_jpg
Binary split	dataset_binary/test/one_hand	135
Binary split	dataset_binary/test/two_hand	215
Binary split	dataset_binary/train/one_hand	605
Binary split	dataset_binary/train/two_hand	1000
Binary split	dataset_binary/val/one_hand	130
Binary split	dataset_binary/val/two_hand	215
Full labeled frame dataset	dataset/one_hand	870
Full labeled frame dataset	dataset/other	110
Full labeled frame dataset	dataset/two_hand	1430
Three-class split	dataset_split/test/one_hand	135
Three-class split	dataset_split/test/other	20
Three-class split	dataset_split/test/two_hand	215
Three-class split	dataset_split/train/one_hand	605
Three-class split	dataset_split/train/other	75
Three-class split	dataset_split/train/two_hand	1000
Three-class split	dataset_split/val/one_hand	130
Three-class split	dataset_split/val/other	15
Three-class split	dataset_split/val/two_hand	215

Exploratory Model Development

The exploratory modeling process went through four phases.

Phase 1: single-frame CNN. I first trained a PyTorch model using a pretrained ResNet-18 where each frame was treated independently:

`image -> one_hand / two_hand / other`

This model reached roughly 60 to 65 percent validation accuracy and about 58 to 63 percent test accuracy depending on the split and configuration. The main insight was that a single frame often does not contain enough information to determine follow-through type. Hand release happens over time, some frames are ambiguous, and the bat path is a motion pattern rather than a static pose.

Phase 2: data refinement. I simplified the classification task by removing the `other` class and training only on one-hand versus two-hand follow-throughs. This made the labels cleaner and the baseball question sharper, but performance improved only slightly, to roughly 63 percent accuracy.

Phase 3: model improvements. I tested standard computer vision improvements, including data augmentation, transfer learning choices, freezing versus fine-tuning layers, and cropping to reduce background noise. These changes produced incremental improvements but no major breakthrough.

Phase 4: multi-frame model. The important modeling change was recognizing that follow-through classification is temporal. I redesigned the input so five RGB frames from the same swing were stacked into one tensor:

`(5 frames) x (3 channels) = 15 input channels`

Instead of a standard (3, 224, 224) image, the model used a (15, 224, 224) input. This allowed the model to see motion progression, hand-release timing, and bat trajectory. The first attempt initially performed poorly, around 55 percent accuracy, because changing the first convolutional layer destroyed the pretrained ResNet input weights. The fix was to copy the pretrained RGB filters into each of the five frame-channel blocks in the new 15-channel convolutional layer. That preserved useful low-level visual features while allowing the model to accept a short sequence.

The final result was exploratory rather than production-ready. The single-frame model generally stayed around 60 to 63 percent accuracy, while the multi-frame model represented the correct direction but was not fully optimized within the project timeline. These results do not have substantial baseball meaning yet; they mainly show that the pipeline can be built and that the problem likely requires temporal information.

Machine Learning Lessons

The machine learning attempt did not become the final source of labels. The major lessons were:

- Problem formulation mattered more than model choice. A temporal problem was initially forced into a static-image setup.
- Data alignment was critical. Much of the project involved matching `playIds`, verifying labels, and correcting ordering issues.
- Transfer learning required care. Changing the model input shape can remove the benefit of pretrained weights if the new first layer is initialized from scratch.
- Baseball domain knowledge mattered. Understanding when the follow-through becomes visible helped diagnose why one frame was insufficient.

The main limitations were:

- The labeled sample was small for image classification.
- Frames from the same swing are highly similar, which makes leakage prevention essential and reduces the effective sample size.
- Broadcast camera angle, player handedness, body occlusion, and timing all add visual noise.
- A single frame often does not contain enough information; a short sequence is better, but harder to model.

- The model learned from Carlson-only labels, so generalizing to other hitters would require more players and more balanced labels.

Even though this did not produce a reliable automated labeler, it is important project work. It shows what would be needed to scale the analysis: a reproducible video pipeline, swing-level splits, multiple frames per swing, a larger manually labeled training set, and likely a true sequence model such as a CNN plus LSTM or a 3D CNN.

Future Computer Vision Work

Future versions of this pipeline should:

- add more players to increase sample size and visual variety
- crop or detect the batter region to reduce background noise
- use a sequence model rather than stacked frames alone
- build a two-stage system: first detect the swing window, then classify the follow-through
- evaluate models at the swing level rather than frame level

Conclusion

The main finding is not simply “two hands good, one hand bad.” The better conclusion is more conditional:

Carlson’s one-hand follow-throughs are concentrated in specific contexts, especially batting side and pitch location. Statcast variables related to location and count explain meaningful variation in follow-through choice. Raw pitch velocity is less important in this sample after those variables are included. When contact quality is modeled separately, follow-through still deserves attention, but the results must be interpreted as association rather than causation because pitch context and swing adjustment are tangled together.

For Carlson specifically, the original baseball recommendation remains plausible: the left-handed swing appears less stable when it finishes with one hand, and many one-hand swings occur on pitches that are less worth attacking. Better plate discipline and a more repeatable two-hand finish may be connected parts of the same improvement path.

The strongest next version of this project would expand the labeled data beyond Carlson. The image-based pipeline here is a start: it successfully connects Statcast data to video, extracts frames, assigns labels, splits data by swing, and trains CNN models. The final accuracy is moderate, but the pipeline demonstrates real-world data engineering, computer vision design, iterative model improvement, and identification of the correct temporal structure of the problem. The statistical analysis would be far more convincing with multiple switch hitters, multiple seasons, and a validated follow-through classifier or larger manual label set.

Appendix: Reproducibility Notes

This report is designed to knit from the files in the project folder. The main required input is:

- `carlson_swings_labeled.csv`

Other relevant files:

- `savant_with_playids.csv`: Statcast data with MLB play IDs.
- `savant_carlson2023.csv`: original Baseball Savant export.
- `OG stuff/Jackson Reel Final Project 2025.ipynb`: original notebook write-up.
- `OG stuff/Dylan Carlson Data.xlsx`: manually collected follow-through data.
- `python scripts/`: video and image-data pipeline scripts.
- `train_model.py`, `test_model.py`, `swing_dataset.py`: image model attempt.